



HW2 Worksheet

Problem Statement

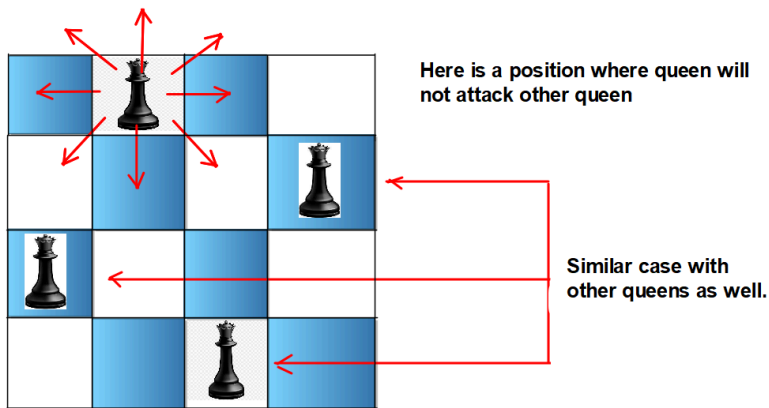
The n-queens problem is a classic computer science question (we call it a search problem in AI). Recall how a queen moves in chess. It can move/attack along rows, columns, and diagonals. The usual problem is to place n queens on an $n \times n$ chessboard such that no two queens attack each other. Check out the following link if you want to read more about the problem:

https://en.wikipedia.org/wiki/Eight_queens_puzzle

We have a twist in this assignment. We will take as input the position of many queens that are already placed on the chessboard, and your code has to place the remaining queens. (Sometimes, this will not be possible.)

Important: the chessboard should be indexed starting from 1, in standard (x, y) coordinates. Thus, (4, 3) refers to the square in the 4th column and 3rd row.

SAMPLE Solution:



Step 1: Manually solve a small example

Solve the following puzzle (one queen, denoted by X is already on the board). **Note that you don't need to solve it with a stack yet.**

	1	2	3	4
1				
2				
3				
4			X	

The Logic/Algorithm

Now write down the algorithm you used to solve the above toy problem in simple words:

Reflection:

1. Did you use a **systematic** way to explore different **options**? **This is the key to formulating a search problem.**
2. Starting from the initial board, what were the **sequences of decisions** you had to make? Did you have to make a decision **per row**? **Per column**? **Per cell**? **Per queen**?
3. How did you **determine** whether a choice is **valid** or not?
4. **When a choice doesn't work down the road**, what do you do? Start over or backtrack to your latest decision point and try a different option? **How do you know which options you have tried before and which ones you haven't**?
5. Did you store your **choices** somewhere (on the paper or in your head) to refer to them later? **Make them explicit. Store them in a stack!**
6. Is your solution efficient? Are you doing any extra work that could have been avoided?

Solve the toy problem again, think “stack”

Solve the above toy problem again using a stack. The stack will store your alternatives so you can use them later when you hit a dead end. Also if those alternatives are not complete yet (still need more queens), they are going to be used for generating new alternatives (making the next choice).

Think about what information you need to pack into each element you store in the stack. You can use tuples to pack different types of info as one stack element.

The very first thing you push in the stack is going to be the root of your search tree. You are going to take it out and apply your first decision to it resulting in several new alternatives that are going to be pushed into the stack for later consideration.

(<info1>, <info2>, ...)

Meet (or exceed) the efficiency requirements

Read all the **instructions** carefully and **write down the efficiency constraints** (if any)

Handling Special Cases and Exceptions:

Can you think of any special cases that need to be handled separately?

Test your code

Now create several test cases (input, correct output), including exceptional/border cases and run your code. If your code fails on any of those test cases, revise your code.

Then try the autograder which will run your code on a few of OUR test cases.

Once you are confident that your code can handle all the test cases, submit it according to the instructions!

